COMP9417 - Machine Learning
Project: Forecasting Electricity Demand and Price Direction under Drift

Project Description

Forecasting electricity demand and prices is central to operating a power system. Market operators and participants rely on these forecasts to plan and schedule generation in advance; if too little generation is available when demand peaks, supply may have to be interrupted. Electricity markets are also a demanding setting for machine learning. Demand and prices change with the weather, the time of day and the season, but also with slow structural changes such as the growth of rooftop solar, the closure of coal plants and shocks to fuel prices. A model that forecasts well when it is deployed can become unreliable a few years later. In this project, you will design, implement and evaluate machine learning models that forecast electricity demand and prices in New South Wales (NSW), and you will study how and why these models degrade over time.

You will work with a dataset built for this course from public data published by the Australian Energy Market Operator (AEMO), combined with historical weather data. It contains half-hourly demand and spot prices for NSW, Victoria and Queensland, together with weather variables for Sydney, Melbourne and Brisbane, from January 2010 to October 2026. Over this period the market has changed considerably, which makes the dataset well suited to studying *distribution shift*: changes over time in the distribution of the inputs, of the targets, or of the relationship between them. Writing P_t for the probability distribution of the data at time t, a change in the relationship between inputs and targets, $P_t(Y \mid X)$, is usually called *concept drift*; a change in the inputs alone, $P_t(X)$, is *covariate shift*, and it does not necessarily harm a model's predictions. Telling these apart is part of the project.

Your task is to build models that forecast the next day's demand and price behaviour, assess how their performance changes as the market evolves, and propose strategies to keep them accurate. At the end of the project, your final models will be evaluated on a week of market data collected after the submission deadline.

Description of the Data

The dataset is provided as a single file, `nem_weather_halfhourly_2010_2026.csv`, with one row for each half-hour from 1 January 2010 to 31 October 2026 (approximately 295,000 records). Table 1 summarises the variables. The file is complete: every half-hour is present, there are no duplicated timestamps and no missing values, so no imputation is needed.

Market variables come from AEMO's aggregated price and demand data; since October 2021 the market settles every five minutes, and these values have been averaged to half-hours so that the whole series is consistent. Prices can be negative, and occasionally spike to several thousand dollars per MWh; these values are genuine and should not be treated as errors. The three states are part of a single market connected by interconnectors, so electricity flows between them and their prices often move together. Victorian and Queensland data may therefore be useful inputs for forecasting NSW; whether they improve your models is for your group to determine and justify.

Weather variables come from Open-Meteo's historical weather API (https://open-meteo.com, CC BY 4.0). These are reanalysis estimates, produced by combining weather models with past observations, rather than direct station measurements, and they may incorporate observations that would not have been available at prediction time. Hourly values have been linearly interpolated to half-hours, so each half-hour value uses the neighbouring hourly values, including the following hour; this is consistent with treating the weather of the forecast day as known.

Table 1: Variables in the dataset.

Variable	Description
`timestamp`	End of the half-hour interval, in market time (AEST, UTC+10, no daylight saving)
`nsw_demand, vic_demand, qld_demand`	Average operational demand in NSW, Victoria and Queensland (MW)
`nsw_price, vic_price, qld_price`	Average spot price in NSW, Victoria and Queensland ($/MWh)
`syd_temp, mel_temp, bne_temp`	Air temperature at 2 m in Sydney, Melbourne and Brisbane (°C)
`syd_rh, mel_rh, bne_rh`	Relative humidity (%)
`syd_cloud, mel_cloud, bne_cloud`	Cloud cover (%)
`syd_solar, mel_solar, bne_solar`	Shortwave solar radiation (W/m^2)

The dataset is fixed. This file is the only data you may use to train models, tune them or engineer features; external data of any kind, including additional AEMO or weather records, is not permitted. The one exception is calendar information: you may use the day of the week, the date, and NSW public holidays from the `holidays` Python package included in the project environment. All groups, including groups with extensions, work with the same data, so that the evaluation described below compares modelling choices rather than data collection.

Prediction Task

At the end of each day D, forecast all 48 half-hours of the following day, $D + 1$. Because timestamps mark the end of each interval, day D consists of the intervals stamped D 00:30 to D 23:30 plus $D + 1$ 00:00. You may use any data up to the end of day D, the weather for day $D + 1$ and calendar information. Demand and prices from day $D + 1$ must never be used as inputs.

The weather variables for day $D + 1$ are historical reanalysis estimates, treated as if they were available at prediction time. This represents an idealised forecasting scenario that separates the machine learning problem from weather-forecasting uncertainty, and it may lead to optimistic estimates of operational per-

formance; you should discuss its implications for real-world deployment.

Let $y_{D,t}$ be NSW demand and $P_{D,t}$ the NSW price in half-hour $t = 1, \ldots, 48$ of day D, and let $\bar{P}_D = \frac{1}{48}\sum_{j=1}^{48} P_{D,j}$ be the mean NSW price of day D. There are two targets:

1. **Demand (regression):** $y_{D+1,t}$, in MW, for each half-hour of $D+1$.
2. **Price direction (classification):** $c_{D+1,t} = \mathbf{1}\left[P_{D+1,t} > \bar{P}_D\right]$, that is, whether the price in a half-hour of $D+1$ is above the previous day's mean price.

The price-direction target is chosen as a well-defined teaching problem rather than as a trading decision. Note that a single extreme price spike on day D raises $\bar{P}_D$ and can push almost every half-hour of $D+1$ into class 0, so the class proportions themselves change over time. This is worth examining in your analysis.

Baselines. Report both baselines throughout:

- *Same half-hour yesterday:* $\hat{y}_{D+1,t} = y_{D,t}$ and $\hat{c}_{D+1,t} = \mathbf{1}\left[P_{D,t} > \bar{P}_D\right]$.
- *Same half-hour last week:* $\hat{y}_{D+1,t} = y_{D-6,t}$ and $\hat{c}_{D+1,t} = \mathbf{1}\left[P_{D-6,t} > \bar{P}_D\right]$.

Both price baselines use the same threshold as the target, $\bar{P}_D$. The same-half-hour-yesterday baseline is the stronger of the two on this data, so it is the reference for the live evaluation. The implementation in the project kit (`harness/dayahead.py`) is the authoritative definition of both baselines and of the targets for marking purposes.

Metrics. Report the following metrics for every evaluated period. They are used to evaluate and compare forecasts; you may train your models with any loss function you consider appropriate. For n half-hours with true demand y_i, predicted demand $\hat{y}_i$, true class c_i and predicted class $\hat{c}_i$:

- Demand: $\mathrm{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{y_i}$ (%) and $\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$ (MW). MAPE is unstable when the true value is close to zero; this is not a concern for NSW operational demand, which stays well above zero.
- Price direction: accuracy, the percentage of half-hours with $\hat{c}_i = c_i$, and balanced accuracy, the mean over the classes present in the period of the recall of each class, where the recall of class k is the percentage of half-hours with $c_i = k$ that are predicted as k. If only one class is present, balanced accuracy equals the recall of that class.

Appendix A uses the same per-half-hour errors to build confidence intervals for monthly MAPE and accuracy. The implementations in `harness/dayahead.py` are authoritative for marking.

Experimental Protocol

Your experiments must follow three stages. At every stage, a forecast for day $D+1$ may use only information available at the end of day D.

Stage 1: Initial model selection. Train on 2010–2013 and use 2014 to select algorithms, features and hyperparameters. Once these are chosen, you may refit the models on 2010–2014. The results are your *frozen* models. For each task, choose one of them as your *selected* model, which is used for the adaptation strategies in Stage 2. All training, validation and test splits, here and throughout the project, must respect the chronological order of the data.

Stage 2: **Historical simulation, January 2015 to October 2026.** Simulate deployment day by day, in chronological order. Evaluate the frozen version of every model you studied in Stage 1, without any updates. Then evaluate your adaptation strategies on the selected model of each task; you may also apply them to other models. Adaptation strategies may only retrain using data available at the time of retraining. Report the frozen and adapted models separately, by year and by month.

Stage 3: **Final model.** Use the results of the historical simulation to choose the strategy for your submitted package, and train the final models on the released data accordingly (you may use any data up to 31 October 2026). Because the simulation results were used to make this choice, they are no longer an unbiased estimate of future performance; the live week is the genuinely unseen evaluation. You are encouraged, but not required, to keep the last part of the simulation (for example, 2025–2026) aside for checking the strategy you select.

Main Objectives

Your work should address the following components:

- **Regression Model Development:** Choose at least two regression algorithms from different families (for example, one linear or kernel-based method, such as regularised Linear Regression, and one tree-based or neural method, such as Gradient Boosting or a Neural Network) to forecast NSW demand. You may study more, but marks reward depth: justify your choice of algorithms and features, including lagged and weather features, and investigate your models in depth rather than adding more models.
- **Classification Model Development:** Choose at least two classification algorithms from different families (for example, Logistic Regression, Naive Bayes, k-Nearest Neighbours, Support Vector Machines, Random Forest, Gradient Boosting or Neural Networks) to forecast price direction. Consider how the changing class proportions affect your models. If an algorithm does not scale to the full dataset, explain how and why you subsample.
- **Model Assessment:** Compare all models against both baselines, and visualise predicted against observed values over time for selected periods.
- **Day-Type Analysis:** Represent each day by its 48 half-hourly NSW demand values. Decide whether to cluster the raw profiles, which mixes the demand level and the daily shape, or profiles normalised by each day's mean, which captures shape only, and justify your choice. Fit PCA, to reduce the dimension of the profiles, followed by a clustering method (e.g., k-means, hierarchical clustering or a Gaussian mixture model) on the training years 2010–2013 only. Then assign every later day to the learned clusters. For this analysis the PCA and the clustering are fitted once and kept fixed, so that later days are compared with a fixed reference. Justify the number of clusters, interpret each cluster, and analyse how the share of each day type changes from year to year. Days unlike any training-period cluster may provide evidence of distribution shift (for example, through their distance to the nearest cluster centre). Investigate whether changes in the share of day types coincide with changes in your models' forecasting error. The main outcome belongs in your report: plots of the typical daily profiles and of their yearly shares, used as evidence of how demand patterns have changed and related to your input-based drift detector. You may also use day types as model features; remember that the type of day $D+1$ is not known when forecasting it. A model that uses them may refit its own clustering when it is retrained, as part of that retraining.

- **Performance Degradation and Drift Detection:**
  1. Plot the error of your frozen models month by month over the historical simulation, with a 95% confidence interval for each month shown as error bars or a shaded band. Use MAPE for demand and accuracy for price direction: accuracy is a simple average of correct/incorrect indicators, so the intervals in Appendix A apply to it directly, whereas balanced accuracy is a combination of two rates computed on subsets of different sizes.
  2. Compute these intervals in two ways, as described in Appendix A: (a) treating each half-hour as an independent observation, and (b) a block bootstrap that resamples whole days. Explain why the two intervals differ in width, what the intervals estimate, and which assumptions each one relies on, including whether whole days can be treated as independent.
  3. Implement one drift detector based only on the input variables and one based on model errors. Each detector must operate online: state the monitored statistic, the reference period, the decision threshold and how the threshold was chosen, using only data that would have been available at the time (for example, choosing thresholds on 2010–2014). Report when each detector signals drift and compare these dates with the observed degradation of your models. Each detector observes day D at the end of day D, and a retraining it triggers takes effect from the forecast for day $D + 1$. Does a change in the inputs always lead to worse predictions? Note that neither input-based nor error-based detection alone establishes whether $P(Y \mid X)$ has changed: an error-based detector may signal because the inputs moved to a region where the model is weaker, and the relationship can change without a large change in average error. Interpreting the signals requires additional analysis. Relate detected changes to plausible events in the electricity market, distinguishing supporting evidence from coincidence in time.
- **Model Adaptation:** For the selected model of each task, compare at least three adaptation strategies (e.g., no retraining, periodic retraining, sliding windows, weighting recent data), including at least one strategy that retrains when one of your drift detectors signals drift. The retraining budget is defined as follows:
  - Each task (demand and price direction) may be retrained at most **six times** between January 2015 and October 2026. The budget applies separately to each combination of model and adaptation strategy that you evaluate.
  - The initial training in Stage 1 does not count towards the budget.
  - A retraining is one occasion on which the deployed model is replaced: any update of fitted parameters, including refitting preprocessing steps such as scalers or clusterings learned from data. Everything done on that occasion counts as a single retraining, including fitting several hyperparameter settings and keeping the best one, provided that the selection uses only past data (for example, validating on the most recent days before the retraining). Computing lagged features or rolling statistics from past data does not count, and neither does updating the internal state of an online drift detector (for example, its running statistics), provided it uses only past data.
  - A retraining at the end of day D may use only data up to the end of day D.

  Analyse the trade-off between the number of retrains and forecasting error, and answer the central question: does detecting distribution shift help decide when to retrain? Justify the strategy you adopt for your final models (Stage 3).
- **Discussion:** Analyse which models and strategies perform best and why. Discuss why some models degrade faster than others, the limitations of your approach, and what would be needed for reliable deployment.

Model Submission and Live Evaluation

In addition to the report, each group submits a model package containing its final, already trained models for both tasks. After the deadline, staff will run every package on the same week of new market data, from 7 to 13 December 2026. This week begins after the latest possible submission date, including extensions, so that all groups are evaluated on identical data that did not exist when the models were submitted. The results will be published on a leaderboard, using a pseudonym chosen by each group.

The package must be a zip file with the following structure:

```
submission.zip
|-- model.py     defines the class Forecaster
|-- train.py     rebuilds artifacts/ from the released CSV file only
`-- artifacts/   fitted models, scalers and settings (maximum 100 MB)
```

The class `Forecaster` must implement the following interface:

```python
import pandas as pd

class Forecaster:
    def __init__(self, artifact_dir: str):
        """Load the fitted models from artifact_dir. No training here. Time limit: 60 s."""

    def predict(self, history: pd.DataFrame, weather_next: pd.DataFrame) -> pd.DataFrame:
        """
        history:      all half-hours from 2010-01-01 00:30 to the end of day D
                      (the interval stamped D+1 00:00),
                      with the same columns as the released CSV, indexed by timestamp.
        weather_next: the 12 weather columns for the 48 half-hours of day D+1.

        Returns a DataFrame with 48 rows and the same index as weather_next:
          nsw_demand_mw  float, predicted NSW demand (MW)
          nsw_price_up   int, 1 if the NSW price is predicted to be above
                         the mean NSW price of day D, else 0
        Time limit: 20 s per call.
        """
```

- **One day per run:** For each live day, a new process creates a `Forecaster` from your `artifacts/` and calls `predict` once. No state is kept between days, and the package must not update its fitted parameters during the live week; `predict` may only compute features (such as lags and rolling statistics) from `history`.
- **Environment:** Packages run on Linux with Python 3.11 and fixed versions of NumPy, pandas, SciPy, scikit-learn, PyTorch (CPU), XGBoost, LightGBM, statsmodels and holidays, listed in `requirements.txt` in the project kit. Create a new Python 3.11 environment for the project and install exactly these versions, as described in the kit's README. You may use any model that runs in this environment. Requests for additional packages can be posted on the course forum until 13 November 2026.
- **Saving models:** Save scikit-learn, XGBoost, LightGBM and statsmodels models with `joblib` or `pickle`, and PyTorch models as a `state_dict` using `torch.save`. Use exactly the library versions in `requirements.txt`, otherwise your models may fail to load. Models may be trained on a GPU, but must run on a CPU.
- **Training script:** `python train.py path/to/nem_weather_halfhourly_2010_2026.csv` must rebuild the contents of `artifacts/` using only the released file, with fixed random seeds, no

network access, and in at most 2 hours on a standard computer (for example, 4 CPU cores and 16 GB of memory; a GPU may be used). It reproduces the submitted models only; the experiments in your report belong in your code submission. Every submission is subject to reproducibility checks, although staff may re-run `train.py` for a sample of groups only.

- **Restrictions:** The package must not access the network or write files outside `/tmp`. Memory is limited to 4 GB.
- **Failures:** Each call must return exactly 48 rows with the correct index, finite values, demand between 0 and 20,000 MW, and `nsw_price_up` equal to 0 or 1. If a call fails, times out, exceeds the memory limit or returns invalid output, that day counts as failed. One failed day is tolerated: that day is filled with the same-half-hour-yesterday baseline and 2 marks are deducted from the live evaluation. A package that fails on more than one day receives no live evaluation marks and is not ranked on the leaderboard.
- **Testing:** Before submitting, run `check_submission.py` from the project kit on the zip you will submit. It runs your package with the same procedure, time limits and output checks used by staff, on the last 14 days of the dataset, and reports any failure. Optional practice runs on 12 and 19 November will evaluate packages uploaded to a practice page on early-November data and publish a practice leaderboard; these runs carry no marks.

Report Structure

Your final report (maximum 10 pages, 11 pt font, 1.5 spacing, excluding references and appendices) should follow this structure:

1. **Introduction:** Problem motivation, dataset description and project goals.
2. **Data Analysis:** Summary statistics, patterns in demand and prices, and the day-type analysis.
3. **Methodology:** Features, prevention of information leakage, the three-stage protocol, baselines and model selection rationale.
4. **Results:** Validation results for both tasks, monthly performance of frozen and adapted models over the historical simulation with 95% confidence intervals, drift detection and adaptation strategies.
5. **Live-Week Forecast:** Your prediction of your package's MAPE and balanced accuracy for 7 to 13 December 2026, with intervals and the reasoning behind them, including what could make the forecast wrong.
6. **Discussion:** Interpretation of results, limitations and improvement strategies.
7. **Conclusion:** Summary of findings and implications.
8. **References:** Include all sources used for models, algorithms or background theory.

All figures must have labelled axes with units and a caption, and must be referred to in the text. Appendices may contain supplementary figures, tables and implementation details, but markers are not required to read them: everything needed to assess your work against the marking criteria must be in the main 10 pages.

Marking Criteria

The project is marked out of 100, as shown in Table 2: 90 marks for the report and 10 marks for the live evaluation of your model package. In addition, the leaderboard awards up to 10 bonus marks. The project mark is capped at 100: bonus marks can make up for marks lost elsewhere in the project, but any excess above 100 is not carried over to any other assessment item. The project contributes 20% of the final course mark. Throughout, marks reward the quality of your reasoning and analysis rather than the number of methods you implement.

Table 2: Marking criteria.

Component	Criterion	Marks	Description
Report	Methodology and baselines	12	Three-stage chronological protocol followed with no information leakage; both baselines implemented as defined; insightful discussion of why the baselines are difficult to beat
Report	Regression models	12	Justified choice of at least two methods from different families; sound feature engineering and temporal validation; systematic comparison against the baselines; insightful analysis of performance differences and degradation
Report	Classification models	12	As for regression, plus a sound treatment of class balance and changing class proportions, and justified subsampling where used
Report	Day-type analysis	10	Clustering fitted on the training years with justified normalisation and number of clusters; clusters interpreted; changes in their prevalence analysed and related to changes in forecasting error; any use as features free of leakage
Report	Degradation and drift detection	15	Monthly error with intervals computed both ways and correctly interpreted, including their assumptions; online detectors with stated reference periods, thresholds and timing; their limits in identifying concept drift recognised; detections compared with the observed degradation; careful, evidence-based links to market events
Report	Model adaptation	12	Retraining budget respected; frozen and adapted models compared; trade-off between retraining and error analysed; evidence on whether drift-triggered retraining helps; final strategy justified
Report	Live-week forecast	10	Your written prediction of your package's live-week performance, with intervals based on your own analysis; marked on the reasoning, not on whether the prediction turns out correct
Report	Presentation	7	Clear structure and writing; figures labelled, captioned and referred to; page limit respected
Model package	Live evaluation	10	Your package's actual performance in the live week, scored automatically against the same-half-hour-yesterday baseline (see below)
	Total	**100**	

Live evaluation (10 marks, scored automatically): The live week tests how well your final models generalise to unseen data; because the models are frozen during the week, it does not test adaptation, for which the historical simulation is the primary evidence. Over the live week, let MAPE_m and BA_m be your package's demand MAPE and price balanced accuracy (in %), and MAPE_b and BA_b those of the same-half-hour-yesterday baseline. Marks are only awarded for doing better than the baseline. With the relative improvement in demand $I = 100\,(1 - \mathrm{MAPE}_m/\mathrm{MAPE}_b)$ and the gain in price balanced accuracy $G = \mathrm{BA}_m - \mathrm{BA}_b$ (percentage points), each task is worth 5 marks:

Marks	Demand: improvement I (%)	Price direction: gain G (percentage points)
5	$I \geq 40$	$G \geq 8$
4	$30 \leq I < 40$	$6 \leq G < 8$
3	$20 \leq I < 30$	$4 \leq G < 6$
2	$10 \leq I < 20$	$2 \leq G < 4$
1	$0 < I < 10$	$0 < G < 2$
0	$I \leq 0$ (no better than the baseline)	$G \leq 0$ (no better than the baseline)

Submitting the baseline itself is allowed, but it earns 0 live evaluation marks. One failed day costs 2 marks (the live evaluation mark cannot fall below 0), and more than one failed day means 0 marks for both tasks. A single week is a noisy test, so results will contain an element of luck; the bands are deliberately coarse for this reason.

Leaderboard bonus (up to 10 marks): Groups whose package is scored (at most one failed day) are ranked against each other, among scored packages only, by the average of their ranks on the two tasks (demand MAPE and price balanced accuracy). The top 5% of groups receive 10 bonus marks, and the next 5% (positions within the top 10%) receive 5 bonus marks; tied groups share a position. Bonus marks count only towards this project, whose mark cannot exceed 100; they are not transferable to other assessment items. Staff may inspect and re-run `train.py` for groups receiving a leaderboard bonus; a package that cannot be reproduced from its `train.py` loses the bonus.

Final exam: One question in the final exam is based on this project. All group members are expected to understand every component of their group's work.

What to Submit

- A **single PDF** report following the structure above.
- A **.zip file with your code** (`.py` files and/or notebooks) used to produce the results in the report. Markers use the code to check that the reported results are genuine.
- Your **model package**, `submission.zip`, as described in *Model Submission and Live Evaluation*.
- When you submit, enter your group's **leaderboard pseudonym** (up to 20 letters, digits or hyphens; no names or zIDs).
- One submission per group. Do not place the PDF inside either zip file.
- Please check Moodle announcements for updates to this spec. It is your responsibility to check for announcements about the spec.

- General questions can be posted in the COMP9417 project discussion forum on Moodle. Please read the existing questions before posting new ones, and only post clarification questions.
- **Use of generative AI.** You may use generative AI tools, such as coding assistants, to help write your code. The report must be written by your group: for the report, you may use standard editing and referencing functions, but not tools that generate or paraphrase text. Whatever tools you use, you are responsible for every line of code and every result you submit, and you should understand them well enough to explain them. You may be asked to account for any part of your submission; if you cannot demonstrate your understanding of it, the matter may be referred to the UNSW Conduct & Integrity Office.
- **Academic integrity.** Code, models and results must not be shared between groups. Packages that use external data, attempt to access hidden data or the network, or interfere with the evaluation system will receive zero for the live evaluation and may be referred for misconduct. General discussion of course concepts is fine, but posting the project, requesting solutions or sharing your solutions on external sites is academic misconduct.

When and Where to Submit

- Due date: **Sunday November 22nd**, 2026 by **6pm** (report, code and model package).
- **Forum support ends at 5pm on Friday November 20th.** The Moodle forum is not actively monitored on weekends, so a question posted on the Saturday or Sunday before the deadline will most likely not be answered before submission closes.
- Work in groups of 4–5 students, registered on Moodle.
- Late submissions will incur a penalty of 5% per day **from the maximum achievable grade**. The project is marked out of 100, so each late day costs 5 marks. Submissions that are more than 5 days late will receive a mark of zero.
- Students granted special consideration may receive an extension of up to 7 days; their models are evaluated in the same live week as all other groups, and must be trained only on the released dataset.
- Submission must be made on Moodle, **no exceptions**.

Peer Review

All group members must complete the peer review survey by **Monday November 23rd**, 2026 at 6pm. Failure to do so will result in a 10% penalty to that student's mark. Peer evaluations will be used to adjust individual grades based on contribution. Groups granted an extension for the project receive the same extension for the peer review survey.

Project Support

The project kit (`comp9417_project_kit.zip`) is available on Moodle. It contains the dataset, a starter notebook that loads the data and computes both baselines, a template and a complete example model

package, the pinned library versions (`requirements.txt`), and `check_submission.py`, which runs your package exactly as in marking. Consult the official documentation of the libraries and the lecture materials for guidance.

Appendix A: Confidence Intervals for Monthly Error

For a given month, let n be the number of half-hours (about 1,440 for a 30-day month). For each half-hour i, let $a_i = 100 \times |y_i - \hat{y}_i|/y_i$ be the absolute percentage error of the demand forecast, and $c_i = 1$ if the price-direction forecast is correct and 0 otherwise. The monthly MAPE is the mean of the a_i, and the monthly accuracy $\hat{p}$ is the mean of the c_i.

What the interval estimates. The monthly MAPE and accuracy are exact descriptions of how the model performed in that particular month. A confidence interval becomes meaningful only when we view the month as one realisation of an underlying random process, and ask about the model's *expected* performance under conditions like those of that month. This interpretation assumes the process is reasonably stable within the month. Not every reported metric needs an interval; here the intervals describe the uncertainty of each monthly performance estimate. Note that comparing two months' intervals (for example, checking whether they overlap) is not a formal test of whether performance changed: that would require an interval for the difference between the two months directly.

(a) Treating half-hours as independent. If the c_i were independent with a common mean p, Hoeffding's inequality would give a distribution-free 95% interval for p:

$$\hat{p} \pm \sqrt{\frac{\ln(2/\delta)}{2n}}, \qquad \delta = 0.05.$$

Hoeffding's inequality requires independent, bounded observations but makes no other assumption about their distribution. Under the same independence assumption, the normal approximation to the binomial distribution gives the narrower interval

$$\hat{p} \pm 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

For MAPE, use the normal approximation for a mean, where s is the standard deviation of the a_i:

$$\text{MAPE} \pm 1.96\,\frac{s}{\sqrt{n}}.$$

For example, with $n = 1{,}440$ the Hoeffding interval for accuracy is about ± 3.6 percentage points, and the binomial interval for an accuracy of 75% is about ± 2.2 points.

(b) Block bootstrap over days. Errors within the same day are strongly correlated: a model that misjudges one afternoon usually misjudges the whole afternoon. The bootstrap therefore resamples whole days rather than individual half-hours:

1. Split the month into its days, keeping all 48 half-hours of each day together.
2. Draw as many days as the month contains, with replacement, from those days.
3. Compute the statistic (MAPE or accuracy) over all half-hours of the drawn days.
4. Repeat steps 2 and 3 at least 1,000 times.
5. Report the 2.5th and 97.5th percentiles of the repeated statistics as the 95% interval.

```python
import numpy as np

def day_block_bootstrap(errors_by_day, stat=np.mean, n_boot=1000, seed=0):
    """errors_by_day: list of arrays, one per day (e.g. the 48 values a_i or c_i)."""
    rng = np.random.default_rng(seed)
    k = len(errors_by_day)
    stats = [stat(np.concatenate([errors_by_day[j] for j in rng.integers(0, k, k)]))
             for _ in range(n_boot)]
    return np.percentile(stats, [2.5, 97.5])
```

What to discuss. Approach (a) assumes the n half-hours are independent, but within-day correlation means a month contains far fewer independent pieces of information than n suggests. The day-block bootstrap preserves this within-day dependence, but it still treats days as independent and interchangeable. Dependence across days remains: consecutive weekdays are similar, and heatwaves or holidays span several days. Compare the widths of the two intervals over the historical simulation, explain the difference, discuss whether longer blocks (for example, whole weeks) would be more appropriate, bearing in mind that a month contains only four or five weeks, which is too few for a reliable bootstrap. Finally, discuss the limitations of interpreting differences between months as evidence of degradation.